

Chapter 1

Abstract Syntax

1.1 Abstract Syntax Trees

Let $\mathcal{S}$ be a finite set of sorts. For a given $\mathcal{S}$, an arity has the form $(s_1, \dots, s_n)s$.

Let $\mathcal{O} = \mathcal{O}_\alpha$ be an arity-indexed family of disjoint sets of operators $\mathcal{O}_\alpha$ of arity α .

Fix a $\mathcal{S}$ and a $\mathcal{O}$. Let $\mathcal{X} = \{\mathcal{X}_s\}_{s \in \mathcal{S}}$ be a sort-indexed family of disjoint finite sets $\mathcal{X}_s$ of variables x of sort s . We say x is fresh for $\mathcal{X}$ if $x \notin \mathcal{X}$ and that $\mathcal{X}, x$ is the set obtained by adding x to $\mathcal{X}_s$ if s is a sort.

Finally, $\mathcal{A}[\mathcal{X}] = \{\mathcal{A}[\mathcal{X}]_s : s \in \mathcal{S}\}$ is the family of **abstract syntax trees** is the smallest family satisfying the following:

1. $\forall x \in \mathcal{X}_s, x \in \mathcal{A}[\mathcal{X}]_s$
2. If o is an operator of arity $(s_1, \dots, s_n)s$ and $a_1 \in \mathcal{A}[\mathcal{X}]_{s_1}, \dots, a_n \in \mathcal{A}[\mathcal{X}]_{s_n}$, then $o(a_1, \dots, a_n) \in \mathcal{A}[\mathcal{X}]_s$

Structural induction can be used to prove that some property $\mathcal{P}(a)$ holds $\forall a \in \mathcal{A}[\mathcal{X}]$ if both of the following hold:

1. Base case: show if $a \in \mathcal{X}_s$, then $\mathcal{P}_s(a)$
2. Inductive case: show if o has arity $(s_1, \dots, s_n)s$ and $\forall i : \mathcal{P}_{s_i}(a_i)$, then $\mathcal{P}_s(o(a_1, \dots, a_n))$

Substitution (denoted with $[b/x]a$ for substituting all occurrences of x with b in a) is defined as

1. $[b/x]x = b$ and $[b/x]y = y$ if $x \neq y$
2. $[b/x]o(a_1; \dots; a_n) = o([b/x]a_1; \dots; [b/x]a_n)$

1.2 Abstract Binding Trees

Abstract binding trees enrich abstract syntax trees by allowing *each argument of an operator* to specify a number of variables to bind. An argument to an ABT operator is called an **abstractor** and has the form $x_1, \dots, x_n.a$

let is a function that can be used to resolve bindings to an abstractor and has the form $\text{let}(a_1, x.a_2)$ or, more naturally, **let** x **be** a_1 **in** a_2 .

ABT operators have **generalized arities** that specify a **valence** for each **abstractor** argument. A valence v has the form $s_1, \dots, s_n.s$ to state that the abstractor binds n arguments of sort $s_1, \dots, s_n$ and is itself of sort s .

The ABTs $\mathcal{B}[\mathcal{X}]$ over a family of sorts $\mathcal{X}$ is defined as

1. If $x \in \mathcal{X}$, then $x \in \mathcal{B}[\mathcal{X}]$
2. For each operator o of arity $(\vec{s}_1.s_1, \dots, \vec{s}_n.s_n)s$, if for each $1 \leq i \leq n$ and each fresh renaming $\rho_i : \vec{x}_i \leftrightarrow \vec{x}'_i$, we have $\hat{\rho}_i(a_i) \in \mathcal{B}[\mathcal{X}, \vec{x}'_i]$, then $o(\vec{x}_1.a_1; \dots; \vec{x}_n.a_n) \in \mathcal{B}[\mathcal{X}]_s$

Similar to ASTs, **structural induction** on ABTs can be used to prove that some property $\mathcal{P}[\mathcal{X}]$ holds for all $a \in \mathcal{B}[\mathcal{X}]$ by showing both:

1. If $x \in \mathcal{X}$ then $\mathcal{P}[\mathcal{X}](x)$
2. For every o of arity $(\vec{s}_1.s_1, \dots, \vec{s}_n.s_n)s$, if for each $1 \leq i \leq n$, $\mathcal{P}[\mathcal{X}, \vec{x}'_i]_{s_i}(\hat{\rho}_i(a_i))$ holds for every $\rho_i : \vec{x}_i \leftrightarrow \vec{x}'_i$ with $\vec{x}'_i \notin \mathcal{X}$, then $\mathcal{P}[\mathcal{X}]_s(o(\vec{x}_1.a_1; \dots; \vec{x}_n.a_n))$

α -equivalence, written $a =_\alpha b$ is a relation that states two ABTs a and b are equivalent up to renaming of names of variables bound within either. α -equivalence is the strongest congruence containing the following conditions:

We will consider α -equivalent ABTs to be the same going forward.

substitution for ABTs is defined as

1. $[b/x]x = b$ and $[b/x]y = y$ (if $y \neq x$)
2. $[b/x]o(\vec{x}_1.a_1; \dots; \vec{x}_n.a_n) = o(\vec{x}_1.a'_1; \dots; \vec{x}_n.a'_n)$ where $\forall i$ we require $\vec{x}_i \notin b$ and we say $a'_i = [b/x]a_i$ if $x \notin \vec{x}_i$ and $a'_i = a_i$ otherwise

The second item in that bullet is to capture that if x is bound within a_i , then occurrences of x within a_i do not refer to the x that the substitution refers to.

One subtlety of substitution is with **capture** of free variables. That is, for $[b/x]o(\vec{x}_1.a_1; \dots; \vec{x}_n.a_n)$ if $y \in \vec{x}_i$, $y \in b$, and $y \neq x$, then the substitution is undefined because y is *captured* by b .

We define **symbols** or **symbolic parameters** in the context of indexing operators. An **indexed operator** o is a family of operators indexed by symbols u such that $o[u]$ is an operator when u is an available symbol. For a finite set of symbols $\mathcal{U}$, $\mathcal{B}[\mathcal{U}; \mathcal{X}]$ is the sort-indexed family of ABTs of $\mathcal{X}$, also admitting all indexed operator instances of symbols $u \in \mathcal{U}$.

Note that while a variable is a place-holder that stands in for an unknown ABT of the given sort, a symbol doesn't stand for anything and is not, itself, an ABT.

We can extend the set of symbols by introducing a fresh symbol within a scope using an abstractor $u.a$ which binds the symbol u within a